

Reviewer Pack — Logarithmic Saturation Gap of Forman-Ricci μ above $v/4$ for F^*_v

Entry 4ccb4473302e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 10:44:02 UTC

Logarithmic Saturation Gap of Forman-Ricci μ above $v/4$ for F^*_v

Entry ID: 4ccb4473302e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 10:44:02 UTC

1. Conjecture

Field A (mathematical branch): Forman-Ricci curvature **Field B** (complexity object): Monotone Boolean functions

Statement:

Let F^*_v denote the k -clique minterm DNF on K_v with $k = \lceil \log_2 v \rceil$, and let $\mu(F^*_v)$ be the orbit-weighted combinatorial Forman-Ricci curvature on its term-overlap graph $N(F_v)$ (one node per k -clique, edge weight $w_{\{a,b\}} = |E(C_a) \cap E(C_b)|$, restricted to $w_{\{a,b\}} \geq 1$). *Conjecture:* for every $v \in \{10, 16, 20, 24, 30, 40\}$ for which the SC5 antecedent $\mu(F_v) \geq v/4$ is empirically verified, the saturation gap satisfies $0.05 \cdot k \leq \mu(F^*_v) - v/4 \leq 5 \cdot k$, i.e. $\mu(F^*_v) - v/4 = \Theta(\log v)$ with explicit constants. A single v in this range with verified antecedent whose gap falls outside $[0.05 \cdot k, 5 \cdot k]$ refutes the conjecture.

Rationale (proposer's reasoning):

If μ genuinely captures the k -clique substructure of F^*_v (the SC5 hypothesis), then the slack above $v/4$ should be governed by the orbit-level combinatorics of pair-intersections, whose dominant contribution scales with $k = \log_2 v$ rather than with v itself. Forman-Ricci curvature aggregates inverse-square-root edge-weight terms that, after summation over the $k+1$ intersection orbits, are known to produce log-order surpluses; pinning the gap to $\Theta(k)$ refines SC5 from a one-sided inequality to a tight asymptotic, isolating clique saturation as the precise source of $v/4$.

Taxonomy category: forman_ricci_dnf (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 08e45c9db257b0ff

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each $v \in \{10, 16, 20, 24, 30, 40\}$ with verified antecedent $\mu(F_v) \geq v/4$, compute $\text{gap}_v = \mu(F^*_v) - v/4$ with $k = \lceil \log_2 v \rceil$ over 30 seeds (200 pairs/orbit). **SUPPORTED** iff every seed for every antecedent-verified v yields $\text{gap}_v/k \in [0.05, 5]$; one violation falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Forman-Ricci curvature clique complex monotone Boolean DNF - combinatorial Forman curvature term overlap graph minterm-orbit-weighted Forman Ricci curvature k-clique logarithmic bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def binomial(n, k):
        if k > n:
            return 0
        if k == 0 or k == n:
            return 1
        k = min(k, n - k)
        c = Fraction(1, 1)
        for i in range(k):
            c *= (n - i) / (i + 1)
        return c

    def forman_ricci_curvature(w_a, w_b, w_ab, w_f):
        if w_ab == 0 or w_f == 0:
            return 0
        term1 = w_a + w_b
        term2 = sum(w_a / math.sqrt(w_ab * w_f) for f in range(n) if (f != a and f != b))
        term3 = sum(w_b / math.sqrt(w_ab * w_f) for f in range(n) if (f != a and f != b))
        return term1 - term2 - term3

    def estimate_mu(v, k):
        n = binomial(v, 2)
        total_orbit_weight = 0
        mu = 0
```

```

for j in range(k + 1):
    0_j = (1 / 2) * binomial(v, k) * binomial(k, j) * binomial(v - k, k - j)
    representatives = random.sample(range(binomial(k, 2)), min(100, binomial(k, 2)))
    for a in representatives:
        for b in range(a + 1, binomial(k, 2)):
            C_a = frozenset((i, j) for i in range(v) if (i, j) in edges and (i, j) not in C_b)
            C_b = frozenset((i, j) for i in range(v) if (i, j) in edges and (i, j) not in C_a)
            w_a = sum(1 for f in range(n) if (f != a and f != b) and (f in C_a or f in C_b))
            w_b = sum(1 for f in range(n) if (f != a and f != b) and (f in C_a or f in C_b))
            w_ab = sum(1 for f in range(n) if (f != a and f != b) and (f in C_a and f in C_b))
            w_f = sum(1 for f in range(n) if (f != a and f != b) and (f in C_a or f in C_b))
            F_j = forman_ricci_curvature(w_a, w_b, w_ab, w_f)
            mu += 0_j * F_j
            total_orbit_weight += 0_j
    return mu / total_orbit_weight if total_orbit_weight != 0 else 0

results = []
for v in [10, 16, 20, 24, 30, 40]:
    k = math.ceil(math.log2(v))
    edges = set()
    for i in range(v):
        for j in range(i + 1, v):
            edges.add((i, j))
    mu = estimate_mu(v, k)
    if mu < v / 4:
        results.append({"v": v, "mu": mu, "conjecture_holds": False})
    else:
        gap = mu - v / 4
        results.append({"v": v, "mu": mu, "gap": gap, "conjecture_holds": 0.05 * k <= gap <= 5})

mean_mu = sum(result["mu"] for result in results) / len(results)
std_mu = math.sqrt(sum((result["mu"] - mean_mu) ** 2 for result in results) / len(results))
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "metric_name": "mu",
    "metric_value": mean_mu,
    "instances_tested": len(results),
    "n_max": max(v for v, _, _ in results),
    "conjecture_holds": support_fraction == 1.0,
    "counterexample": "" if support_fraction == 1.0 else f"v={results[0]['v']}, mu={results[0]['mu']}"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")

    mean_mu = sum(result["mu"] for result in results) / len(results)
    std_mu = math.sqrt(sum((result["mu"] - mean_mu) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    print(f"RESULT: SUPPORTED mean={mean_mu} std={std_mu} support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_af43a571.py", line 90, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_af43a571.py", line 67, in run_trial
    mu = estimate_mu(v, k)
          ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_af43a571.py", line 46, in estimate_mu
    representatives = random.sample(range(binomial(k, 2)), min(100, binomial(k, 2)))
                                   ~~~~~
TypeError: 'float' object cannot be interpreted as an integer
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` before producing any data (binomial returned a float passed to `range()`), so no gap values were computed and the pre-registered support condition cannot be evaluated. | next: Fix the test by casting `binomial(k,2)` to `int` (or use `math.comb`) and rerun the 30-seed sweep across $v \in \{10,16,20,24,30,40\}$.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	63268
2	preregistration	claude_max	opus	0	0	7073
3	novelty	claude_max	opus	0	0	3316
4	novelty	claude_max	opus	0	0	5593
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	123136
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14733
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12193
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15034
9	judge	claude_max	opus	0	0	4639

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 248986 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/4ccb4473302e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4ccb4473302e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4ccb4473302e.tar.gz` (if generated)